

Task

Create a minimum functional e-commerce web application using **React + TypeScript**.

The application must run locally at:

```
http://localhost:8080
```

This is a **frontend-only application**. Do not create or require a backend server.

Use the attached UX reference images and API contracts as the source of truth.

General Requirements

- Use React.
 - Use TypeScript.
 - Application must run on `localhost:8080`.
 - No backend/server implementation is required.
 - Use only the APIs supplied in the attached API contracts.
 - Do not search for, replace, or introduce alternative APIs.
 - Follow the supplied UX designs as closely as reasonably possible.
 - Keep the application minimal. Do not add unnecessary functionality.
 - Handle API and application errors gracefully.
 - Follow accessibility best practices.
 - Follow frontend performance best practices.
 - Follow applicable SEO best practices.
 - Use responsive layouts so the application remains usable at different screen sizes.
-

1. Login

Use the supplied **Login UX** and **Login API Contract**.

The page must contain:

- Username
- Password
- Login button

The password field should support show/hide password.

Use:

```
POST https://dummyjson.com/auth/login
```

Use the request and response structure defined in the supplied Login API Contract.

The supplied test account is:

- Username: `emilys`
- Password: `emilyspass`

On successful authentication:

- retain the authenticated user's required session information;
- retain the returned user ID for subsequent cart operations;
- navigate to the products page.

Handle:

- empty username;
- empty password;
- invalid credentials;
- API/network failure;
- loading state while authentication is in progress.

The API contract specifies the authentication endpoint, supplied credentials, user ID and returned authentication tokens.

2. Product Listing

Use the supplied **Product Listing UX** and **Products & Reviews API Contract**.

Retrieve products using:

```
GET https://dummyjson.com/products
```

Display multiple products in a responsive grid.

For each product display:

- product image;
- title;
- description;
- price;
- discount percentage;
- rating;
- review control/count;

- brand;
- Add to Cart button.

Use the API fields specified in the supplied contract:

- `id`
- `title`
- `description`
- `price`
- `discountPercentage`
- `rating`
- `brand`
- `thumbnail`
- `reviews`

The product contract explicitly defines these fields and requires reviews associated with each product to be used for that product.

Handle:

- products loading;
- failed API request;
- empty product result;
- missing optional product information;
- missing/broken product image.

Do not hard-code product data from the UX image. The UX image defines the visual layout; actual product content must come from the API.

3. Product Reviews

Use the supplied **Review Modal UX**.

When the user selects the review/rating control for a product, open a modal showing the reviews belonging to that product.

Each available review should display:

- reviewer name;
- rating;
- comment;
- review date.

Use:

- `reviews[].reviewerName`

- `reviews[].rating`
- `reviews[].comment`
- `reviews[].date`

from the Products API. These review fields are provided directly by the product API contract.

The modal must:

- clearly identify itself as a dialog;
- be keyboard accessible;
- allow the user to close it;
- support closing with the Escape key;
- manage focus appropriately;
- prevent inaccessible interaction with background content while open.

If a product has no reviews, display an appropriate empty state.

Do not invent reviews.

4. Add to Cart

When the user selects **Add to Cart**, use the supplied Add to Cart API contract.

API:

```
POST https://dummyjson.com/carts/add
```

Build the request using:

- authenticated `userId`;
- selected product `id`;
- selected quantity.

The cart contract requires product IDs and quantities and returns product/cart totals and quantity information.

Maintain the cart state in the frontend so users can continue adding products during the current application session.

The cart icon in the top navigation must display the number of items currently in the cart.

Handle Add to Cart API failures without losing the existing valid cart state.

5. Cart / Order Screen

Selecting the cart icon should open the cart/order page shown in the supplied UX reference.

Display each cart item with:

- product image;
- title;
- price;
- quantity;
- item total;
- remove option.

Allow the user to:

- increase quantity;
- decrease quantity;
- remove an item;
- continue shopping.

Update totals whenever quantities change.

Display an order summary containing at minimum:

- subtotal;
- total;
- Place Order button.

Use cart response information where applicable, including:

- returned products;
- total;
- discounted total;
- total products;
- total quantity.

The supplied cart API returns these values explicitly.

No actual payment processing is required.

A successful Place Order action may show an order-success confirmation within the frontend application.

6. Accessibility

The application must follow reasonable WCAG-oriented frontend practices.

At minimum:

- use semantic HTML;
 - associate labels with form fields;
 - support keyboard navigation;
 - provide visible focus indicators;
 - provide meaningful button and link names;
 - provide image alternative text;
 - provide accessible error messages;
 - do not rely only on color to communicate information;
 - use appropriate dialog semantics for the review modal;
 - provide appropriate ARIA attributes only when native HTML semantics are insufficient.
-

7. Error Handling

Provide user-friendly states for:

- login failure;
- network failure;
- product API failure;
- cart API failure;
- empty product results;
- missing image;
- no reviews;
- invalid form input.

Do not expose raw stack traces or technical exceptions in the UI.

Errors should not cause the entire application to become unusable when a localized fallback is possible.

8. Performance

Apply reasonable React/frontend performance practices.

Examples include:

- avoid unnecessary component re-renders;
- use appropriate component boundaries;
- avoid unnecessary dependencies;
- optimize image loading;
- lazy-load where appropriate;
- avoid unnecessary API requests;
- use stable keys when rendering lists;

- keep the production bundle reasonably small.

Do not prematurely add complex optimization that is unnecessary for this application.

9. SEO

Implement reasonable SEO practices applicable to this client-side e-commerce application.

At minimum:

- meaningful page title;
- useful meta description;
- semantic heading hierarchy;
- descriptive image alt text;
- meaningful document structure.

Do not introduce a server-side rendering framework solely for SEO. The required technology remains React + TypeScript running as a frontend-only application.

10. Code Quality

Follow standard React + TypeScript practices.

The implementation should:

- use TypeScript types/interfaces for API data;
 - separate API logic from presentation where reasonable;
 - avoid duplicated logic;
 - use reusable components where appropriate;
 - avoid unnecessary abstraction;
 - avoid `any` where reasonable;
 - keep components readable and maintainable;
 - handle asynchronous operations correctly;
 - clean up side effects when required.
-

UX Requirement

The attached UX images define the expected appearance and interaction model for:

1. Login
2. Product listing

- 3. Review modal
- 4. Cart/order page

Reproduce these designs closely, but use actual API data rather than the sample content visible in the images.

Do not redesign the application or introduce additional pages/features unless required for the functionality specified above.

Completion Criteria

The task is complete when a user can successfully perform this flow:

Login → View Products → View Product Reviews → Add Product(s) to Cart → Open Cart → Modify Cart → Place Order

The complete application must run at:

```
http://localhost:8080
```

No separate backend service should be required.